transforms, writes the data, and finishes before the next scheduled execution.

It's like doing laundry: instead of washing each item immediately, you wait for a full load, making the process more efficient. The trade-off is that new data isn't processed until the next batch.

Batch jobs are typically scheduled using tools like Cron, orchestrated with Airflow, and processed by platforms such as Spark or dbt. Since the system processes the entire dataset at once, it can optimise execution efficiently.

Batch processing is ideal for financial reporting, machine learning model training, ETL pipelines, and analytics where real-time updates aren't required. When yesterday's data is sufficient, batch processing is usually the most cost-effective, reliable, and easy-to-maintain approach.

What streaming means in practice

Streaming takes the opposite approach to batch processing by handling each event as it occurs instead of waiting to process data in bulk. Every click, transaction, or update is processed within milliseconds, enabling immediate responses.

A good analogy is a busy restaurant kitchen, where orders are prepared as they arrive rather than waiting for a large batch. Streaming systems continuously process an endless flow of events, requiring a different architecture and mindset than batch processing.

The biggest advantage is low latency. Dashboards can display near real-time data, fraud detection can stop suspicious transactions instantly, and inventory updates can sync across locations without delay.

However, streaming is more complex. It must manage event ordering, late-arriving data, exactly-once processing, state management, and failure recovery. While it delivers real-time insights, it is more difficult and costly to build, operate, and maintain than batch processing.

Where the line actually sits

Here's the question I always ask first when somebody is debating between batch and streaming. How fresh does the data need to be?

If the answer is "within a few hours is fine," batch is your friend. Build the pipeline once, schedule it, walk away. You'll thank yourself later.

If the answer is "within a few minutes," you're in the middle ground. Micro-batching, where you run small batch jobs every few minutes, often works well here. It gets you most of the freshness benefits of streaming with most of the operational simplicity of batch.

If the answer is "within seconds," you genuinely need streaming. There's no shortcut. Use a real stream processing system and pay the complexity tax.

The mistake I see most often is teams jumping to streaming because it sounds modern and impressive, when their actual business need was perfectly served by an hourly batch job. Streaming introduces real engineering overhead. Make sure you need it before you take that on.

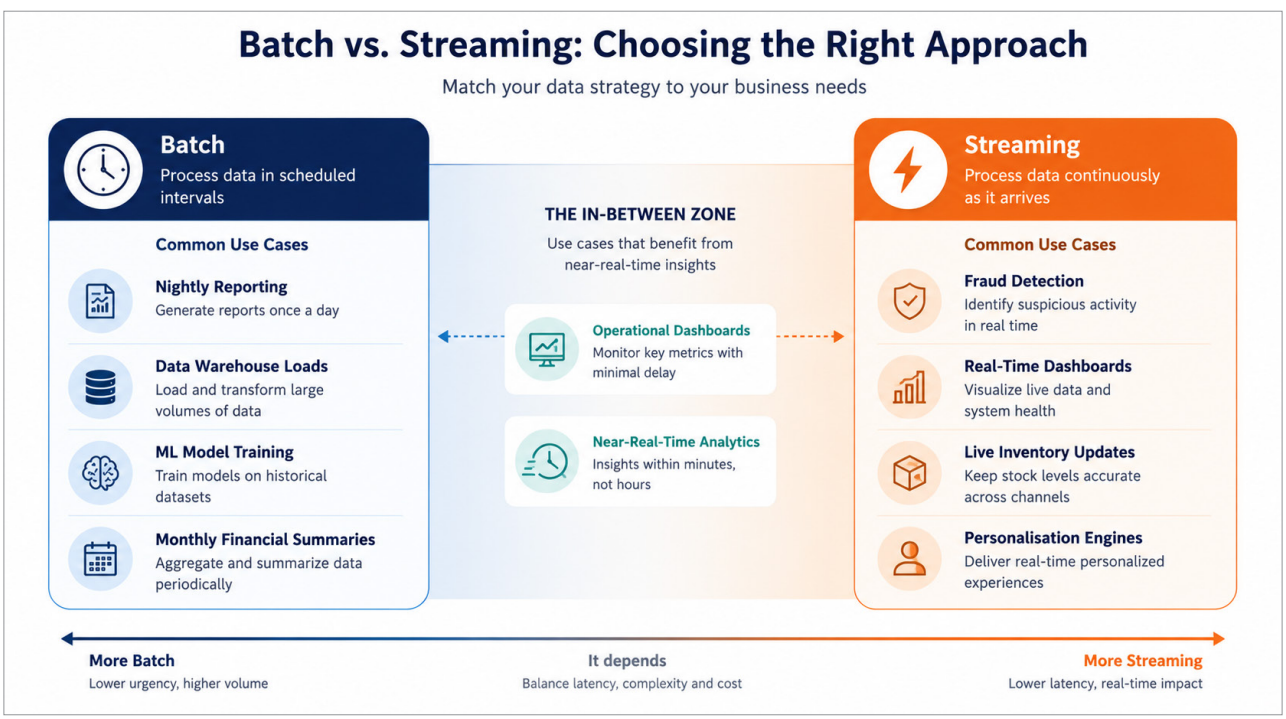

Figure 1: Batch vs streaming